Homework 2 (Agentic AI): Building a Q&A Agent for Chemicals Using SMILES Codes

You will be assigned a group of 3 students based on a variety of factors. Only 1 student from the group of 3 needs to submit on Canvas.

Your Task

You will build a Q&A agent for chemicals using SMILES codes. The user of the chatbot will, via free-text conversation, ask the chatbot queries about the SMILES string of chemicals. Your Q&A agent should take a question from a user and, based on the question, provide the answer to the following queries about a SMILES string:

- Molecular Weight
- Lipophilicity / Hydrophobicity (logP)
- Topological polar surface area (TPSA)
- Number of hydrogen bond donors
- Number of hydrogen bond acceptors
- Number of rotatable bonds
- Number of rings
- Number of aromatic rings

If the user asks about any other task (excluding the two extra functionalities you will add) or doesn't provide a SMILES code, then the agent should tell the user it only answers the 8 questions listed above (or, if no SMILES was given, ask for a valid SMILES string).

Additionally, you must add two additional useful features to your agent – of your choosing. This can range from RAG, additional MCP features beyond the `RDKit` library ones that are required, functionality for calling leading drug discovery tools, adding memory (e.g., if you say "*How about for CCO?*", the agent will know what you want it to do based on the last thing you typed), or anything else of your choosing.

You will complete the assignment by modifying the `Homework2-TEMPLATE.py` file. This file has an infinite loop that repeatedly asks the end user to write a prompt, which is then fed into the 4 billion parameter version of Qwen 3. The file currently just prints Qwen 3's response.

Your task is to modify this script such that:

- If the end user asks for one of the 8 chemical queries listed above and provides a valid SMILES string, then the chatbot should respond with the answer to the query about the SMILES string, ideally by calling the `RDKit` library (all required `RDKit` functions are already imported for you). The response should be provided in a user-friendly manner. Bonus points if the response mimics the tone of the user. Importantly, the user should be able to ask the question in any style or format.
- If the SMILES string is invalid, the chatbot should tell the end user to provide a valid SMILES string.
- If the end user asks a question or enters any text not related to the 8 tasks listed above, the agent should respond to the user telling them that it only answers the 8 questions listed above.
- You must add two additional useful features to your agent – of your choosing. This can range from RAG, additional MCP features beyond the `RDKit` library ones that are required, functionality for calling leading drug discovery tools, adding memory (e.g., if you say "*How about for CCO?*", the agent will know what you want it to do based on the last thing you typed), or anything else of your choosing.

Your chatbot should be robust enough to handle multiple queries in one prompt. Your chatbot should be robust enough to handle spelling errors.

Importantly, the agent should respond in the same way regardless of how the prompt is asked. For example, your agent should respond to the following prompts in the same way:

- "Calculate the molecular weight of CCO and logP of c1ccccc1."
- "Do you happen to know the molecular weight of CCO and the logP of c1ccccc1?"
- "Please, tell me: mol. weight of CCO. Also, I'd like to know the lipophilicity of c1ccccc1."
- "what is molecular wght of CCO.....and lphilicity of::: c1ccccc1??????!!!!!!!"

Getting Started

You are provided with three starter files:

- `Homework2-TEMPLATE.py` – this is the file that you will modify and submit.
- `environment.yml` – this has the correct package versions to use for installation to enable Peter to grade your agent properly
- `DemoWhyPromptingAloneIsInsufficient.py` – this is a similar script to the template file, without a loop. Try asking for the sample tasks just by prompting the

Qwen model. You will see why a harness engineering approach is necessary to make this work.

Everyone should work from the same pinned environment so that results are comparable. Run the following commands in your terminal:

```
conda env create -f environment.yml
```

```
conda activate cdd203-hw2
```

You will fill in two TODOs in the assignment: one for the system prompt going into the Qwen model, which analyzes the user prompt, and one for outputting the correct response. You will likely need to change much more of the code, especially for your two extra features. These two TODOs are there to guide you in where to add the code for the `RDKit` library functionality. But feel free to modify the file as you see fit.

Notes

When using HuggingFace to download a model, it will take longer to run on the very first try (like several minutes or even 1 hour), but subsequent runs will be much quicker.

If your extra features need packages beyond `environment.yml`, add them to an `environment.yml` in your repo and mention it in your `README.md` file. However, if using these extra packages, be sure to consult with Peter first for approval!

Submission

Submit on Canvas a link to a GitHub repository containing:

1. Python script (.py) called `Homework2.py` that contains your Chemical Information Chatbot Agent. Your code should automatically download and call whatever model you are using, so do not include model weights or hard-coded local file paths in your repository.
2. A short writeup (up to 2 paragraphs) explaining (a) your strategy for designing your system prompt(s) you used to help process the user's input and (b) how you ultimately routed the user's text to tool calls. Put this paragraph in the `README.md` file in your GitHub repository.
3. A short writeup (about 2 paragraphs total, 1 paragraph per feature) explaining (a) the two additional features you added, (b) the technical details of how these additional features were added, and (c) how to activate these features in the agent (i.e., how Peter can try out the features when grading). Put this paragraph in the `README.md` file in your GitHub repository.
4. A short paragraph explaining how you used an LLM/generative AI to help with your assignment. Put this paragraph in the `README.md` file in your GitHub repository.

All group member names should be in the `README.md` file.

Grading

- Passing hidden held-out test cases: 55%
- Two additional features added to the agent, of your choosing: 20% (10% each)
- Writeup about design decisions: 10%
- Writeup about your two additional features: 10%
- Reflection on LLM use: 5%
- Extra credit of up to 5% if the agent's response mimics the tone of the user when providing the answers, to a reasonable degree. This will be evaluated subjectively.
- Extra credit of up to 5% if the agent uses an MCP server for the `RDKit` library tool calls, to mimic best practices.